

Q&A — 평판분석 / 정적분석 모듈 동작 설명

발표 후 Q&A 대비용. 질문별로 1-2분 답변 분량으로 정리.

Q1. "평판분석 모듈 어떻게 동작해요?"

한 줄 답변: "패키지 자체의 코드는 안 보고, 외부에 공개된 신뢰 신호들을 수집해서 verifier에게 evidence로 전달합니다."

아티팩트별로 어떤 소스를 보는지 (총 4 모듈):

모듈	소스	신호
pypi_reputation	OSV.dev · deps.dev · PyPI JSON API	CVE 목록 (버전 필터링됨) · 패키지 나이 ...
npm_reputation	OSV.dev · npmjs.org downloads · deps.dev	CVE · 주간 다운로드 수 (typosquat...
repo_reputation	OpenSSF Scorecard · GitHub API · GHSA Ad...	osb score · stars/forks/l...
skill_reputation	파일 경로 · SKILL.md frontmatt...	distribution channel (anth...

핵심 디자인 결정 (질문 깊게 들어올 때):

- OSV는 버전 인지 (version-aware) —
reputation/_osv.py::_pypi_range_covers가 PEP 440 events[] (introduced/fixed/last_affected)를 읽고 pip install이 받게 될 최신 버전에 영향 주는 CVE만 통과. Django 5.0 → OSV 300건이 33건으로 줄어듦.
- 공유 헬퍼 _osv.py — pypi와 npm 모듈이 같은 쿼리 로직 쓰는데 캐싱(_CACHE)도 공유. OSV API 호출 줄이고 결과 일관성 보장.
- known-bad 리스트는 1차 출처 인용 의무 —
_known_bad.py 헤더에 DataDog/OSSF/OSV/GHSA/SkillSieve 모두 URL+라이선스 기재. 출처 없는 차단리스트는 변호 불가능하다는 원칙.
- 두 개 독립 출처 동시 확인 — _ossf_malicious.py가 OSSF malicious-packages 서브트리 별도 로드해서 DataDog corpus와 교차 검증.

****평판분석이 *안* 하는 것**:** 코드 내용 분석. 그건 정적분석 책임.

Q2. "정적분석 모듈 어떻게 동작해요?"

한 줄 답변: "패키지 소스를 받아서 semgrep 기반 룰 체인으로 의심 패턴을 찾고, 거기에 분석 회피(obfuscation) 휴리스틱과 install-hook 검사를 더해 evidence로 만듭니다."

아티팩트별 분석기 (총 4 모듈):

모듈	핵심 동작
pypi_analyzer	semgrep 4-를 체인 (p/security-audit + Guard...
npm_analyzer	semgrep (GuardDog npm 룰 위주, p/security-a...
repo_analyzer	pypi 체인 그대로 delegate (언어-agnostic 패턴만으로도...
skill_analyzer	명령어 표면(SKILL.md/manifest)에 phrase 매칭 + 실...

핵심 디자인 결정:

- Finding category 분류
(pypi_analyzer.categorize_finding) — 각 finding을 install_time (setup.py / __init__.py / pyproject.toml / package.json 같이 *설치 도중* 실행되는 파일에서 나온 신호) vs use_time (그 외 일반 *.py에서 나온 신호)으로 태그. Verifier prompt가 install_time은 무겁게, use_time은 컨텍스트로만 가중. "보안 취약(insecurity) ≠ 악성(maliciousness)" 경계 — CHASE 논문이 자기들 FP 원인으로 명시한 것과 같은 framing.
- GuardDog 룰셋을 벤더링(external_rules_guarddog/) — 외부 의존성 없이 재현 가능. unscoped variant는 path filter 무시하고 모든 *.py에 적용.
- Obfuscation 휴리스틱 (_obfuscation.py) — semgrep과 독립적인 시그널. 파일 크기 단독은 false-positive (legitimate 모듈도 큼) → bytes-per-line ratio가 진짜 discriminator (정상 30-80, 패킹된 페이로드 수천). Shannon entropy는 실험 후 폐기 (정상 5.0-5.6, EZBEAMER 페이로드는 반복 패턴이라 3.02 — 양쪽 다 못 가름).
- npm_npm_manifest.scan_install_hooks — GuardDog의 npm-install-script 룰이 path glob 때문에 scan root의 package.json을 놓치는 버그 우회. 로컬에서 결정론적 검사. 빌드 툴(node-gyp, husky 등)은 allowlist.
- Skill cross-file walk — obvious_injections 1-3 케이스가 SKILL.md가 아니라 ooxml.md에 페이로드 심음. SKILL.md만 보면 못 잡음 → instruction surface에서 언급된 모든 상대 경로를 따라 들어가서 그 파일도 evidence에 포함.

Q3. "왜 평판 + 정적 두 개로 나뉘어요?"

답변: "각 모듈의 reliability 주장을 좁고 검증 가능하게 유지하려고. 평판 모듈은 *외부 출처*에 대한 신뢰성 (OSV가 살아있나, deps.dev 응답이 맞나) 만 책임지고, 정적 분석은 *코드 내용*에 대한 reliability (semgrep 룰이 패턴을 잡나, obfuscation 휴리스틱이 false-positive 없나) 만 책임짐. 두 신호가

verifier에서 합쳐져서 최종 결정."

벤치마크도 이 분리에 맞춰 따로 측정함:

- reputation_reliability.py — 평판 모듈만 격리해서 evaluator (OSV mock 등)
 - static_analysis_reliability.py — 정적 모듈만 격리해서 evaluator
 - framework_reliability.py — 전체 파이프라인 (평판+정적+verifier+sandbox) 통합
-

Q4. "평판 vs 정적 어느 게 더 중요한 신호예요?"

답변: "공격 패턴마다 다름."

- typosquat: 평판이 결정적 (이름은 닮았는데 다운로드 0, 패키지 나이 1일 → 정적은 아직 패턴 못 잡을 수 있음)
- install-time RCE: 정적이 결정적 (setup.py에 `os.system(curl ... | sh)` → 평판은 신규 패키지면 무해해 보임)
- 타입 + 페이로드 결합: 둘 다 봐야 함. 그래서 verifier가 두 신호를 모두 받아서 가중치 결정.

실제 비율 (FP-fix 후 census 기준): 차단 결정의 70% 정도가 정적 신호 단독, 20%가 평판 단독, 10%가 양쪽 confluent. Recall 95.4%에 양쪽이 동시에 기여.

Q5. "왜 LLM 한 번만 호출해요? CHASE는 4-agent 쓰는데"

답변: "latency vs reliability 트레이드오프. CHASE는 케이스당 3-5분, chanever는 45-90초. Multi-agent 체인이 가져다 주는 self-correction을 포기하는 대신 verifier한테 들어가는 evidence package를 풍부하게 만들었음 — CHASE의 Web Researcher가 런타임에 찾으려 가는 정보 (OSV, deps.dev, downloads, Scorecard) 를 우린 evidence 단계에서 미리 다 모아놓음."

이게 우리 F1이 CHASE보다 0.33pp 낮은 이유 중 하나 (98.86 vs 99.19). 대신 recall 100% (CHASE 98.4%) — 평판 신호 덕분에 다운로드 0인 신규 악성을 CHASE보다 잘 잡음.

Q6. "verifier가 어떻게 결정해요?"

(만약 들어오면)

답변: "evidence package — 평판 신호 + 정적 finding

리스트 + (있으면) 동적 sandbox trace — 가 Claude verifier한테 JSON으로 넘어가고, prompt에 명시된 threat-model 규칙 (install_time 무겁게, use_time 가볍게, uncertainty 시 hold 등)에 따라 allow / hold / block 중 하나 출력. 룰 기반 차단은 873c4cf 커밋에서 다 제거하고 LLM 판단에 위임."